

IoT Data Stream to the Cloud using AWS IoT Core

PROJECT REPORT

Submitted by

Hima. L

Department of Computer Science and Engineering (CSE)

Don Bosco Institute of Technology

Program: Cloud Computing

Technologies Used

Python • MQTT • AWS IoT Core • Amazon DynamoDB • Amazon CloudWatch

1. Introduction

Internet of Things (IoT) systems collect data from sensors and transmit it to cloud platforms for storage, processing, monitoring, and analysis. This project demonstrates an IoT data stream from a simulated temperature and humidity sensor to the AWS cloud using AWS IoT Core. The sensor data is published through MQTT, processed by an AWS IoT Rule, stored in Amazon DynamoDB, and monitored using Amazon CloudWatch.

2. Objective

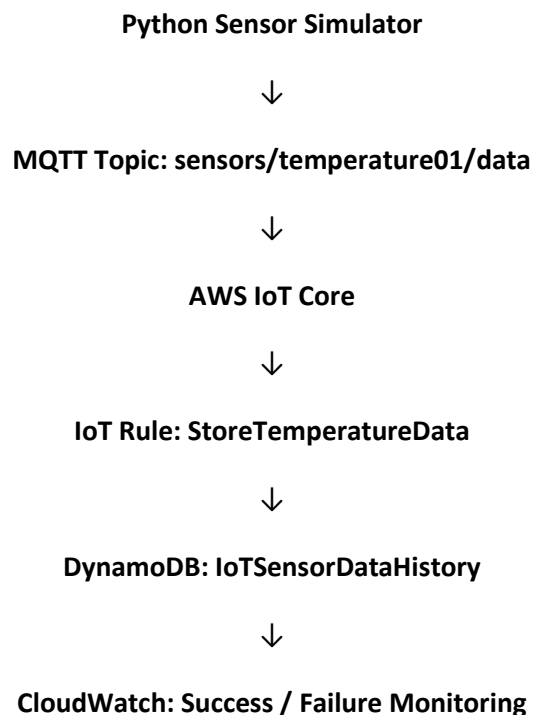
The main objective of this project is to implement and verify an end-to-end IoT data streaming pipeline using AWS services. The system is designed to generate sensor readings, securely publish them to AWS IoT Core, route the messages using an IoT Rule, store the readings in DynamoDB, and monitor rule execution using CloudWatch.

3. Technologies and AWS Services Used

- Python – used to simulate temperature and humidity sensor readings.
- MQTT – used as the lightweight messaging protocol for sensor data.
- AWS IoT Core – receives and manages MQTT sensor messages.
- AWS IoT Rule – processes incoming messages and routes them to DynamoDB.
- Amazon DynamoDB – stores current and historical sensor readings.
- Amazon CloudWatch – monitors IoT rule action success and failure metrics.

4. System Architecture

The implemented architecture follows the data path below:



5. Sensor Data Generation

A Python simulator was created to generate realistic test readings for TemperatureSensor01. Temperature and humidity values were generated continuously and published every five seconds.

MQTT Topic: sensors/temperature01/data

Each JSON sensor message contains:

- device_id
- temperature
- humidity
- timestamp

Example message:

```
{
  "device_id": "TemperatureSensor01",
  "temperature": 26.47,
  "humidity": 75.93,
  "timestamp": "2026-08-30T13:28:53.666745+00:00"
}
```

6. AWS IoT Core Configuration

AWS IoT Core was configured to receive the sensor messages using an MQTT topic. A device certificate and private key were used by the Python MQTT client to establish a secure connection with AWS IoT Core. The IoT policy was configured to permit the required connection and publishing operations.

7. AWS IoT Rule Configuration

An IoT Rule named StoreTemperatureData was created and kept active. The rule reads messages from the sensor MQTT topic and selects the fields required for storage. A DynamoDBv2 action was configured to store the processed data.

```
SELECT device_id, temperature, humidity, timestamp
FROM 'sensors/temperature01/data'
```

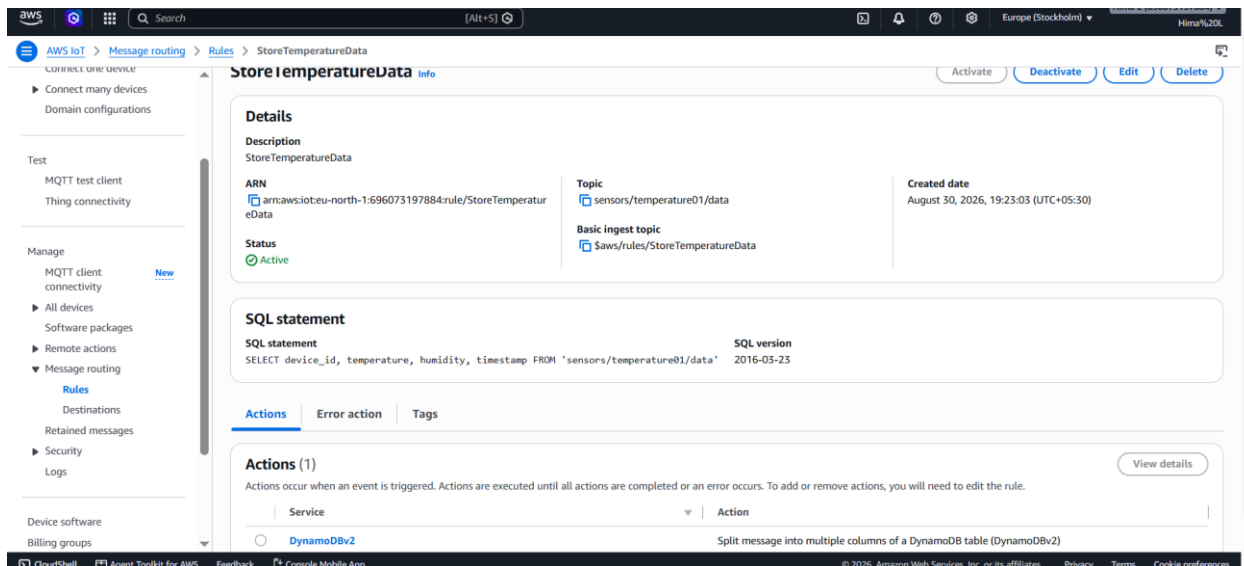


Figure 1. AWS IoT Rule StoreTemperatureData showing the active rule, MQTT topic, SQL statement, and DynamoDBv2 action.

8. Data Storage in Amazon DynamoDB

Two DynamoDB tables were used in the implementation. IoTSensorData stores the latest sensor reading, while IoTSensorDataHistory stores historical sensor readings using device_id as the partition key and timestamp as the sort key. During testing, the history table successfully accumulated sensor records; the captured evidence shows 50 returned items.

- Current data table: IoTSensorData
- Historical data table: IoTSensorDataHistory
- Partition key: device_id
- Sort key for history: timestamp

The screenshot shows the AWS DynamoDB console for the 'IoTSensorDataHistory' table. The table contains 50 items. The columns are device_id, timestamp, humidity, and temperature. The data is as follows:

device_id (String)	timestamp (String)	humidity	temperature
TemperatureSensor01	2026-08-30T14:00:00...	65.2	30.5
TemperatureSensor01	2026-08-30T14:28:32...	73.17	30.22
TemperatureSensor01	2026-08-30T14:28:37...	66.12	25.27
TemperatureSensor01	2026-08-30T14:28:42...	71.71	33.94
TemperatureSensor01	2026-08-30T14:28:47...	62.13	27.78
TemperatureSensor01	2026-08-30T14:28:52...	58.35	34.18
TemperatureSensor01	2026-08-30T14:28:57...	54.65	25.2
TemperatureSensor01	2026-08-30T14:34:31...	69.99	28.1
TemperatureSensor01	2026-08-30T14:34:36...	60.05	33.56
TemperatureSensor01	2026-08-30T14:34:41...	54.02	25.83

Figure 2. DynamoDB IoTSensorDataHistory table showing stored TemperatureSensor01 readings with timestamp, humidity, and temperature values.

9. CloudWatch Monitoring

A CloudWatch dashboard named IoTSensorDashboard was created to monitor the IoT Rule action. Two metrics were added: Success and Failure. The Success metric records successful rule action executions, while the Failure metric is used to identify unsuccessful executions. During verification, successful events were observed and the Failure metric remained at zero.

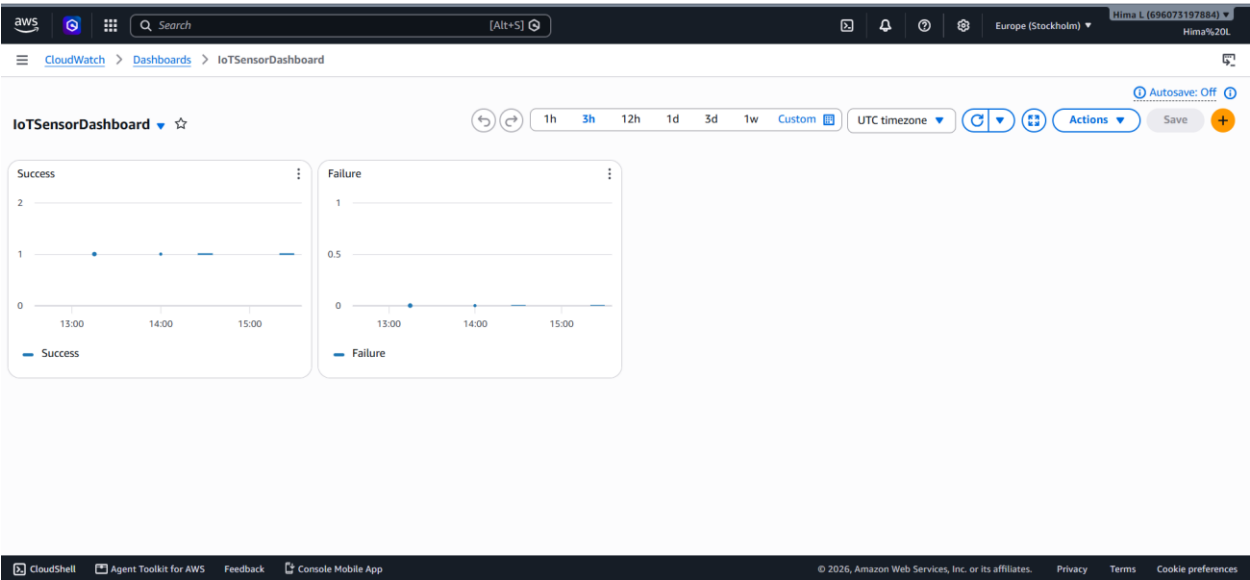


Figure 3. CloudWatch IoTSensorDashboard showing Success and Failure metrics for the IoT rule action.

10. Testing and Verification

The system was tested by running the Python simulator and observing the incoming MQTT messages in the AWS IoT MQTT test client. The sensor messages were then verified in DynamoDB. The historical item count increased during testing, demonstrating continuous data ingestion. CloudWatch also recorded successful IoT rule actions and no observed failures during the final verification period.

Test	Expected Result	Observed Result
MQTT publish	Sensor JSON reaches AWS IoT Core	Successful
IoT Rule	Incoming message is processed	Successful
DynamoDB storage	Sensor records are stored	Successful; 50 records shown in evidence
CloudWatch Success	Successful rule actions are recorded	Observed
CloudWatch Failure	No failures during normal operation	0 observed in final verification

11. Results

The project successfully established an end-to-end IoT data stream to the cloud. The Python simulator generated temperature and humidity data and published it over MQTT. AWS IoT Core received the messages and the StoreTemperatureData rule routed the required fields to DynamoDB. The DynamoDB history table accumulated sensor records, and CloudWatch provided monitoring of successful and failed rule actions.

12. Evidence Summary

Figure	Evidence	Purpose
Figure 1	AWS IoT Rule	Proves the rule is active and routes MQTT data to DynamoDBv2.
Figure 2	DynamoDB History	Proves sensor records are stored successfully.
Figure 3	CloudWatch Dashboard	Proves rule action monitoring through Success and Failure metrics.

13. Conclusion

The project successfully demonstrates how IoT sensor data can be streamed to the cloud using AWS IoT Core. A Python sensor simulator generated temperature and humidity readings, MQTT transported the data, AWS IoT Core processed the messages through an IoT Rule, DynamoDB stored the sensor information, and CloudWatch monitored rule execution. The successful test results confirm that the complete data pipeline operates as intended.

14. Final Verification Checklist

- ✓Python sensor simulator generated temperature and humidity readings.
- ✓MQTT topic sensors/temperature01/data was used for sensor messages.
- ✓AWS IoT Core received the sensor messages.
- ✓StoreTemperatureData IoT Rule was configured and active.
- ✓DynamoDBv2 action was configured for the IoT Rule.
- ✓IoTSensorData stored the latest sensor reading.
- ✓IoTSensorDataHistory stored historical sensor readings.
- ✓CloudWatch IoTSensorDashboard displayed Success and Failure metrics.
- ✓Evidence screenshots were captured for the IoT Rule, DynamoDB, and CloudWatch.
- ✓Python simulator was stopped after testing.